

1. Final AWS Architecture:

High-Level Flow

EC2 (Airflow Scheduler)



Fetch + Process Data



S3 (Data Lake Storage)



Lambda (Model Inference / Trigger)



S3 (Predictions / Models)



2. AWS Services:



EC2 → Compute Layer

- Runs:
 - Apache Airflow
 - Your DAG (`gold_war_pipeline_v3`)
- Why EC2:
 - Full control over environment
 - Needed for Airflow scheduler + dependencies



S3 → Storage Layer

- Stores:
 - gold_prices.csv
 - war_news.csv
 - training_data.csv
 - snapshots/
 - models/

- Why S3:
 - Cheap
 - Scalable
 - Durable (11 9's)

Lambda → Serverless Layer

- Used for:
 - Model inference (optional)
 - Trigger post-processing
 - Lightweight evaluation

3. Data Flow (Cloud Version)

Step-by-step pipeline:

Step 1: Airflow (EC2)

- Runs DAG weekly
- Executes:
 - `fetch_gold_prices()`
 - `fetch_war_news()`
 - `compute_sentiment_and_merge()`
 - `train_model()`

Step 2: Store to S3

Replace local paths with:

 **S3 Bucket:** `gold-war-pipeline-bucket`

`s3://gold-war-pipeline-bucket/`

```
|
|
|— raw/
|   |— gold_prices.csv
|   |— war_news.csv
|
```

- └─ processed/
 - └─ training_data.csv
-
- └─ snapshots/
 - └─ training_data_20260316.csv
 - └─ training_data_20260317.csv
 - └─ training_data_20260323.csv
 - └─ ... (weekly snapshots auto-generated)
-
- └─ models/
 - └─ gold_model.pkl (latest production model)
 - └─ gold_model_20260316.pkl (versioned)
 - └─ gold_model_20260317.pkl
 - └─ gold_model_20260323.pkl
 - └─ ... (future versions)
-
- └─ predictions/
 - └─ predictions_20260316.csv
 - └─ predictions_20260317.csv
 - └─ predictions_20260323.csv
 - └─ ... (generated by Lambda)
-
- └─ logs/ (optional but recommended)
 - └─ airflow/
 - └─ dag_runs/
 - └─ task_logs/
 - └─ lambda/

```
|
└─ config/ (optional but production-grade)
    ├── features.json
    ├── parameters.json
    └─ pipeline_config.yaml
```

Step 3: Lambda Trigger

Triggered by:

- New model upload OR
- New training data

Lambda:

- Loads latest model
- Runs quick prediction / validation
- Stores output back to S3



4. Implementation Plan (Step-by-Step)

-  **Step 1: Create S3 Bucket**
- Name: “gold-war-pipeline-bucket”

Structure:

raw/

processed/

snapshots/

models/

predictions/

◆ Step 2: Launch EC2 Instance

Recommended:

- Type: `t3.micro`
- OS: Ubuntu 22.04

Install:

- `sudo apt update`
- `sudo apt install python3-pip -y`
- `pip install apache-airflow pandas yfinance feedparser textblob scikit-learn joblib boto3`

◆ Step 3: Modify Airflow Code for S3

Replace local saving:

```
df.to_csv(GOLD_FILE)
```

With:

```
import boto3
```

```
s3 = boto3.client('s3')
```

```
s3.upload_file(str(GOLD_FILE), 'gold-war-pipeline-bucket', 'raw/gold_prices.csv')
```

👉 Do this for:

- `gold_prices.csv`
- `war_news.csv`
- `training_data.csv`
- `models`

◆ Step 4: IAM Role (CRITICAL)

- Attach to EC2:
- Permissions:
 - I. `AmazonS3FullAccess`
 - II. `AWSLambdaFullAccess`

◆ Step 5: Create Lambda Function

Runtime: Python 3.10 or higher.

Function:

```
import boto3
import joblib
import pandas as pd
import os

def lambda_handler(event, context):
    s3 = boto3.client('s3')

    bucket = "gold-war-pipeline-bucket"

    # Download model
    s3.download_file(bucket, "models/gold_model.pkl", "/tmp/model.pkl")
    model = joblib.load("/tmp/model.pkl")

    # Download latest data
    s3.download_file(bucket, "processed/training_data.csv", "/tmp/data.csv")
    df = pd.read_csv("/tmp/data.csv")

    X = df.tail(1)[[
        "price_change", "ma7", "ma14",
        "high_low_diff", "open_close_diff",
        "volume", "sentiment_mean", "news_count"
    ]]

    pred = model.predict(X)[0]

    return {"prediction": int(pred)}
```

◆ Step 6: Trigger Lambda

Options:

Option A (Best):

- S3 trigger: models/gold_model.pkl uploaded → trigger Lambda

Option B:

- Airflow triggers Lambda using boto



5. Cost Optimization:



Use Free Tier:

- EC2: t3.micro
- S3: minimal storage
- Lambda: free 1M requests



Stop EC2 when not used



Use lifecycle rules in S3:

- Move old snapshots → Glacier



6. Scalability Design:

- EC2 can be replaced with:
 - 👉 Managed Airflow (MWAA)
- Lambda auto-scales:
 - 👉 Handles spikes automatically
- S3 scales infinitely:
 - 👉 No storage concern



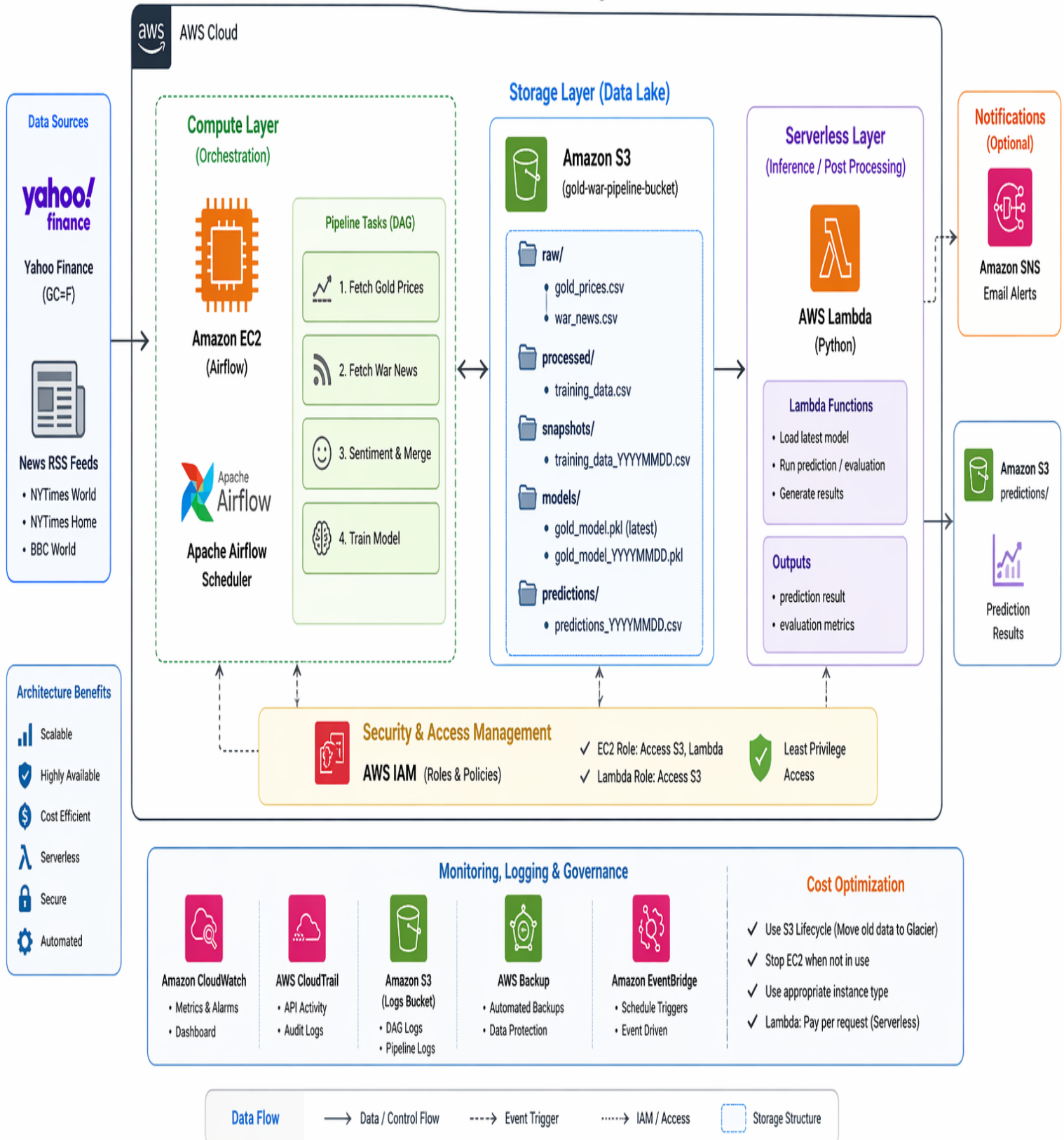
8. Architecture Justification:

- EC2 → needed for Airflow control
- S3 → central data lake
- Lambda → event-driven compute
- IAM → secure access
- Modular → easy to extend

9. Architecture Diagram:

Gold Price & War News ML Pipeline on AWS

Cloud Architecture Diagram



1. Final Event-Driven Flow

```
Airflow (EC2)
  ↓
Upload model to S3 (models/gold_model.pkl)
  ↓
S3 Event Trigger
  ↓
AWS Lambda executes
  ↓
Prediction saved to S3 (predictions/)
```

2. What We Are Triggering

Trigger condition:

```
S3 Object Created
Prefix: models/
File: gold_model.pkl
```

Meaning:

- Every time **latest model is updated**
 - Lambda runs automatically
-

3. Configure S3 Event Trigger

Step-by-step (Console)

Go to:

```
S3 → gold-war-pipeline-bucket → Properties
```

Scroll to:

```
Event notifications → Create event notification
```

Configuration:

Name:

```
trigger-lambda-on-model-upload
```

◆ **Event type:**

✓ Object creation (All)

◆ **Prefix:**

models/

◆ **Suffix:**

gold_model.pkl

◆ **Destination:**

👉 Select your Lambda function

4. Lambda Function (Production Version)

Use this improved version:

```
import boto3
import joblib
import pandas as pd
import os

s3 = boto3.client("s3")

BUCKET = "gold-war-pipeline-bucket"

def lambda_handler(event, context):
    try:
        # Step 1: Download latest model
        s3.download_file(BUCKET, "models/gold_model.pkl", "/tmp/model.pkl")
        model = joblib.load("/tmp/model.pkl")

        # Step 2: Download latest data
        s3.download_file(BUCKET, "processed/training_data.csv", "/tmp/data.csv")
        df = pd.read_csv("/tmp/data.csv")

        # Step 3: Take latest row
        latest = df.tail(1)
```

```

FEATURES = [
    "price_change", "ma7", "ma14",
    "high_low_diff", "open_close_diff",
    "volume", "sentiment_mean", "news_count"
]

X = latest[FEATURES]

# Step 4: Predict
pred = model.predict(X)[0]

result = latest.copy()
result["prediction"] = int(pred)

# Step 5: Save prediction
output_path = "/tmp/prediction.csv"
result.to_csv(output_path, index=False)

s3.upload_file(
    output_path,
    BUCKET,
    "predictions/latest_prediction.csv"
)

return {
    "status": "success",
    "prediction": int(pred)
}

except Exception as e:
    return {
        "status": "error",
        "message": str(e)
    }

```

5. IAM Permissions (CRITICAL)

Lambda Role must have:

AmazonS3FullAccess

(Production version → restrict to bucket)

6. Important Gotchas

! Lambda storage limit

- /tmp = 512 MB
 - Your model is small → OK
-

! Cold start

- First execution slower
 - Fine for this use case
-

! Duplicate triggers

- Multiple uploads → multiple executions
 - Acceptable OR control via naming
-

7. Output in S3

```
predictions/  
  latest_prediction.csv
```

Future improvement:

```
predictions/  
  prediction_20260323.csv
```

